

UNIVERSIDAD SERGIO ARBOLEDA

**LABORATORIO DE BUSQUEDA INFORMADA CON PAC-MAN:
PARTE 3 – BUSQUEDA DE ALIMENTOS (ACTIVIDADES 10 Y 11)**

Inteligencia Artificial (SIST5036)

Wiston

*Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingenieria
Bogota D.C., 2026*

Índice

1. Actividad 10. Busqueda de todos los alimentos	2
1.1. El problema	2
1.2. Por qué crece tanto el espacio de estados	2
1.3. Procedimiento y resultados	2
2. Actividad 11. Diseno de heurísticas para la busqueda de alimentos	3
2.1. Heurística 1: distancia al alimento más lejano	3
2.2. Heurística 2: diámetro Manhattan de posición + comida restante	3
2.3. Optimización con <code>problem.heuristicInfo</code> : qué se cachea y por qué . .	5
2.4. Procedimiento y resultados	5
2.5. El reto del caché: qué encontramos (y por qué es un resultado honesto, no un error)	5

Nota de alcance

Este documento cubre únicamente la Parte 3 del trabajo grupal (Actividades 10 y 11: búsqueda de todos los alimentos y diseño de heurísticas), asignada a Wiston. Se entrega por separado para facilitar la unificación posterior con las Partes 1 (Actividades 1-6) y 2 (Actividades 7-9, problema de las esquinas) de los demás integrantes del grupo en el informe final único. Todo el código (`search.py`, `searchAgents.py`) y los experimentos citados aquí se incluyen junto a este documento, listos para ejecutarse de forma aislada.

1 Actividad 10. Búsqueda de todos los alimentos

1.1 El problema

`FoodSearchProblem` ya viene completamente implementado en `searchAgents.py` (`getStartState`, `isGoalState`, `getSuccessors` y `getCostOfActions`): no hubo que escribir código nuevo en esta actividad. El objetivo es entender el problema y establecer una línea base antes de diseñar heurísticas en la Actividad 11.

El estado ya no es solo la posición de Pac-Man: es el par $s = (p, F)$, donde p es la posición (x, y) y F es una `Grid` (matriz booleana) con el estado –presente o consumido– de cada alimento del laberinto:

```
def isGoalState(self, state):
    return state[1].count() == 0
```

La meta se alcanza cuando ya no queda ningún alimento por consumir (`foodGrid.count() == 0`), sin importar en qué posición esté Pac-Man.

1.2 Por qué crece tanto el espacio de estados

Cada alimento puede estar en dos condiciones: presente o consumido. Si el laberinto tiene F alimentos, el número de configuraciones posibles del `Grid` de comida es, conceptualmente, 2^F –y el estado completo es el producto de eso por el número de posiciones posibles de Pac-Man–. Esto es exactamente lo que dice la guía, y lo confirmamos experimentalmente: intentamos correr `uniformCostSearch` sobre `smallClassic` (55 alimentos) con un límite de 45 segundos y **no terminó**. 2^{55} es un número astronómicamente más grande que cualquier espacio de estados manejado en las Actividades 2-9 (donde el estado era solo una posición, o una posición más 4 bits de esquinas visitadas – $2^4 = 16$ combinaciones como máximo). Por esta razón, esta actividad y la 11 se trabajan sobre `tinySearch` (1 alimento) y `testClassic` (8 alimentos), donde UCS puro sigue siendo viable como línea base.

1.3 Procedimiento y resultados

Se ejecutaron UCS y A* con `nullHeuristic` ($h(n) = 0$) sobre `tinySearch` y `testClassic` (ver `experimentos/actividad10_food_baseline.py`), replicando la misma verificación de la Actividad 4 (con $h(n) = 0$, A* debe coincidir exactamente con UCS) pero ahora sobre `FoodSearchProblem`.

Cuadro 1: Línea base de UCS y A*+h(n)=0 sobre FoodSearchProblem

Layout (alimentos)	Método	Costo	Expandidos	Tiempo (s)
tinySearch (1)	UCS	8	16	0.000393
tinySearch (1)	A*+h(n)=0	8	16	0.000303
testClassic (8)	UCS	16	2598	0.063502
testClassic (8)	A*+h(n)=0	16	2598	0.061340

En ambos layouts, UCS y A*+h(n)=0 encuentran el mismo costo óptimo y expanden exactamente el mismo número de nodos –la misma consecuencia algebraica de $f(n) = g(n) + 0 = g(n)$ ya explicada en la Actividad 4–. Nótese el salto de 16 a 2598 nodos expandidos al pasar de 1 a solo 8 alimentos: es la primera evidencia numérica, dentro de un caso todavía manejable, de la explosión combinatoria que describe la guía, y es exactamente la motivación para diseñar una heurística no trivial en la Actividad 11 –sin ella, layouts con más alimentos que testClassic dejan de ser viables, como mostró el intento fallido sobre smallClassic.

2 Actividad 11. Diseño de heurísticas para la búsqueda de alimentos

2.1 Heurística 1: distancia al alimento más lejano

Es exactamente la fórmula que sugiere la guía como primera aproximación: $h(n) = \max_{f \in F} d_M(n, f)$, la distancia Manhattan desde la posición actual hasta el alimento restante más lejano.

```
def foodHeuristicV1(state, problem):
    position, foodGrid = state
    foodList = foodGrid.asList()
    if not foodList:
        return 0
    return max(abs(position[0] - f[0]) + abs(position[1] - f[1])
               for f in foodList)
```

Admisibilidad: el costo real para recoger toda la comida restante es, como mínimo, el costo de llegar hasta el alimento más lejano –en algún momento del recorrido Pac-Man tiene que llegar hasta él–, y la distancia Manhattan nunca sobreestima una distancia real en una cuadrícula con paredes. Por lo tanto $h(n) \leq h^*(n)$ siempre.

Consistencia: para cualquier sucesor n' de n (un paso ortogonal, costo 1), si el alimento más lejano de n sigue presente en n' (no fue el que se acaba de comer), la desigualdad triangular de Manhattan da $d_M(n, f^*) \leq d_M(n, n') + d_M(n', f^*) = 1 + d_M(n', f^*) \leq 1 + h(n')$. Si el alimento más lejano de n era justo el que estaba en n' (se acaba de comer), $d_M(n, n') = 1 \leq 1 + h(n')$ porque $h(n') \geq 0$ siempre. En ambos casos $h(n) \leq 1 + h(n')$.

2.2 Heurística 2: diámetro Manhattan de posición + comida restante

Generaliza la Heurística 1: en vez de mirar solo la distancia de Pac-Man al alimento más lejano, considera el **diámetro** (la mayor distancia Manhattan entre cualquier par) del

conjunto formado por la posición actual *más* todos los alimentos restantes:

$$h(n) = \max \left(\max_{f \in F} d_M(n, f), \max_{f_i, f_j \in F} d_M(f_i, f_j) \right)$$

Es más informativa que la Heurística 1 porque también considera que, para visitar dos alimentos lejanos entre sí, Pac-Man tiene que recorrer al menos la distancia real entre ellos en algún momento del camino –no solo la distancia desde su posición actual hasta el más lejano de los dos–.

```
def foodHeuristic(state, problem):
    position, foodGrid = state
    foodList = foodGrid.asList()
    if not foodList:
        return 0

    if 'foodPairDistances' not in problem.heuristicInfo:
        allFood = problem.start[1].asList()
        distancias = {}
        for i in range(len(allFood)):
            for j in range(i + 1, len(allFood)):
                f1, f2 = allFood[i], allFood[j]
                d = abs(f1[0]-f2[0]) + abs(f1[1]-f2[1])
                distancias[(f1, f2)] = d
                distancias[(f2, f1)] = d
        problem.heuristicInfo['foodPairDistances'] = distancias

    distanciasComida = problem.heuristicInfo['foodPairDistances']

    maxDist = 0
    for f in foodList:
        d = abs(position[0]-f[0]) + abs(position[1]-f[1])
        if d > maxDist:
            maxDist = d
    for i in range(len(foodList)):
        for j in range(i+1, len(foodList)):
            d = distanciasComida[(foodList[i], foodList[j])]
            if d > maxDist:
                maxDist = d
    return maxDist
```

Admisibilidad: el recorrido que visita, en cualquier orden, la posición n y todos los alimentos de F tiene una longitud real que es, como mínimo, la distancia real entre el par de puntos (de $\{n\} \cup F$) más separado entre sí –en algún momento del recorrido Pac-Man pasa por ambos puntos de ese par–. Como la distancia Manhattan nunca sobreestima la distancia real, el diámetro en Manhattan de $\{n\} \cup F$ es una cota inferior válida: $h(n) \leq h^*(n)$.

Consistencia: el mismo argumento de desigualdad triangular de la Heurística 1 se aplica aquí a cualquier par de puntos de $\{n, n'\} \cup F$. El caso más delicado es cuando se acaba de comer el alimento en n' (es decir, $F' = F \setminus \{n'\}$ tras el paso): como $\{n\} \cup$

$F = \{n, n'\} \cup F'$, para cualquier par que incluya a n y a algún $q \in \{n'\} \cup F'$ se cumple $d_M(n, q) \leq d_M(n, n') + d_M(n', q) = 1 + d_M(n', q) \leq 1 + h(n')$ (porque $q \in \{n'\} \cup F'$ acota por el máximo); si el par no incluye a n , ya está dentro de $\{n'\} \cup F'$ y su distancia es, por definición, $\leq h(n')$. Por lo tanto $h(n) \leq 1 + h(n')$ en todos los casos, y $h(\text{meta}) = 0$ porque con $F = \emptyset$ el conjunto $\{n\}$ es un único punto (diámetro 0).

2.3 Optimización con `problem.heuristicInfo`: qué se cachea y por qué

El cálculo que se repite en *cada* llamada a la heurística (una por cada nodo expandido, potencialmente miles de veces) es la distancia Manhattan entre cada *par* de alimentos. Pero las posiciones de los alimentos nunca cambian durante la búsqueda –solo si siguen presentes o no–, así que esas distancias se calculan una sola vez (para todos los pares del layout inicial, usando `problem.start[1]`, el `foodGrid` original) y se guardan en `problem.heuristicInfo['foodPairDistances']`; las llamadas siguientes reutilizan ese diccionario en vez de recalculer la fórmula de distancia para el mismo par de alimentos.

2.4 Procedimiento y resultados

Se ejecutó A* con las tres heurísticas ($h(n) = 0$, Heurística 1, Heurística 2) sobre `tinySearch` y `testClassic` (ver experimentos/actividad11_food_heuristic.py).

Cuadro 2: Comparación de heurísticas de comida sobre `testClassic` (8 alimentos)

Heurística	Costo	Longitud	Expandidos	Tiempo (s)
$h(n) = 0$	16	16	2598	0.070222
Heurística 1	16	16	702	0.024714
Heurística 2	16	16	383	0.014872

En `tinySearch` (1 alimento) las tres coinciden en costo (8) y las dos heurísticas empatan en 8 nodos expandidos frente a los 16 de $h(n) = 0$ –con un solo alimento no hay ninguna pareja de alimentos que comparar, así que la Heurística 2 se reduce exactamente a la Heurística 1–. La diferencia real aparece en `testClassic`: la Heurística 1 ya reduce drásticamente la exploración frente a $h(n) = 0$ ($2598 \rightarrow 702$, factor $R \approx 3,70$), y la Heurística 2 la reduce aún más ($2598 \rightarrow 383$, factor $R \approx 6,78$; 319 nodos menos que la Heurística 1), confirmando que considerar también la distancia entre pares de alimentos –no solo la distancia desde Pac-Man– aporta información adicional real, no solo teórica.

2.5 El reto del caché: qué encontramos (y por qué es un resultado honesto, no un error)

Comparamos el tiempo de A*+Heurística 2 con caché (`foodHeuristic`) contra una versión idéntica pero sin caché (`foodHeuristicV2SinCache`, que recalcula las distancias entre pares de alimentos restantes en cada llamada), promediando 5 corridas por versión en dos layouts:

Cuadro 3: Heurística 2 con caché vs. sin caché (tiempo promedio de 5 corridas)

Layout (alimentos)	Expandidos	Tiempo con caché (s)	Tiempo sin caché (s)
testClassic (8)	383	0.015458	0.014835
capsuleClassic (23)	2185	0.207050	0.193121

El resultado es contraintuitivo pero reproducible: la versión *sin* caché fue, en ambos casos, ligeramente *más rápida* (entre 4% y 7%), no más lenta. La explicación no es que el caché esté mal implementado –los nodos expandidos son idénticos en ambas versiones, porque es exactamente la misma heurística–, sino que el cálculo que se está cacheando es demasiado barato para que valga la pena: son sumas y restas de enteros ($\text{abs}(f1[0]-f2[0]) + \text{abs}(f1[1]-f2[1])$), mientras que consultar el diccionario `problem.heuristicInfo['foodPairDistances']` implica construir una tupla (`f1`, `f2`) como llave y calcular su hash en cada consulta. En este problema, con a lo sumo un par de decenas de alimentos, el costo de esa búsqueda en el diccionario termina siendo comparable –o incluso mayor– al costo de simplemente recalcularla resta. El caché con `problem.heuristicInfo` sigue siendo la herramienta correcta que sugiere la guía, pero su beneficio depende de que el cálculo evitado sea realmente costoso (por ejemplo, una distancia real de laberinto calculada con BFS en vez de una resta Manhattan); aquí, honestamente, no lo es. Dejamos ambas versiones en el código (`foodHeuristic` con caché se usa en `AStarFoodSearchAgent`, y `foodHeuristicV2SinCache` existe solo para esta comparación) precisamente para poder mostrar esta medición en vivo si se pide en la sustentación.